

O Especialista em Claude Code e o Fundador que Cansou de Dizer "Continua"

Transcrição da prática 1 · 14 de setembro de 2026, 16:03

Rodrigo Sampaio 00:00

"E aí! Olha, eu uso o Claude Code pra refatorar e migrar código, mas é exaustivo: ele faz um pedaço, trava e para, e eu fico a noite inteira digitando 'continua'. Me falaram do /goal, que deixa ele trabalhando sozinho até o fim.

Só que bate o medo: como ele sabe que realmente terminou? E se entrar num loop e torrar meu orçamento enquanto eu durmo?"

O medo do loop infinito é super justo! Deixar um agente de IA rodando sozinho no terminal dá aquele frio na barriga de acordar com uma fatura astronômica. [1] (<https://medium.com/@0xmega/claude-code-is-eating-your-budget-7-fixes-that-cut-costs-without-killing-output-f8dbb3b8a04d>) Mas a boa notícia é que o comando `/goal` foi projetado com mecanismos de segurança bem específicos para evitar que isso aconteça. [1] (<https://prospere.ai/en/blog/claude-goal-one-command-forces-ai/>), [2] (<https://code.claude.com/docs/en/goal>) Aqui está uma explicação simples de como o `/goal` funciona nos bastidores e como você pode se proteger para dormir em paz. [1] (https://www.reddit.com/r/ClaudeAI/comments/1tatxau/claude_code_just_shipped_a_run_until_done_mode/)

Como o `/goal` sabe que realmente terminou?

O Claude Code não usa apenas o bom senso para decidir se acabou; ele usa uma arquitetura de dupla validação: [1] (<https://medium.com/@vasu7yadav/the-goal-command-is-the-most-important-agent-primitive-of-2026-1ef4188448b1>), [2] (https://www.reddit.com/r/ClaudeCode/comments/1tmm4sd/how_i_ran_a_9hour_autonomous_goal_session_with/)

O Executor (Claude Principal): Ele planeja, mexe nos arquivos, roda comandos no terminal e tenta resolver o problema.

O Avaliador (Subagente): Toda vez que o Claude principal termina uma rodada (um "turno"), entra em ação um segundo modelo em segundo plano (geralmente o Haiku, que é super rápido e barato). Esse subagente lê o histórico e avalia friamente: "A condição que o usuário pediu foi atingida de forma verificável?" Se a resposta for não, ele dá o feedback do que falta e manda o executor continuar. Ele só para quando o objetivo é atingido de forma concreta. [1] (https://www.youtube.com/watch?v=59zh_m2WD-8), [2] (<https://www.mindstudio.ai/blog/claude-code-goal-command-autonomous-tasks>), [3] (<https://www.mindstudio.ai/blog/claude-code-goal-loop-commands-autonomous-tasks>), [4] (<https://prospere.ai/en/blog/claude-goal-one-command-forces-ai/>), [5] (https://www.reddit.com/r/ClaudeCode/comments/1tmm4sd/how_i_ran_a_9hour_autonomous_goal_session_with/)

Os 3 Escudos Antiloop (Para você não falir)

O Claude Code tem travas nativas para impedir que ele gaste todo o seu orçamento: [1] (<https://code.claude.com/docs/en/goal>), [2] (<https://medium.com/@0xmega/claude-code-is-eating-your-budget-7-fixes-that-cut-costs-without-killing-output-f8dbb3b8a04d>)

Detector de Falta de Progresso: Se o Claude passar vários turnos seguidos sem usar ferramentas (ou seja, só "conversando" com o avaliador sem de fato alterar arquivos ou rodar comandos), o Claude Code para o loop automaticamente, joga um aviso na tela e devolve o controle para você. [1] (<https://code.claude.com/docs/en/goal>)

Limite de "Check-ins" Ociosos: Enquanto o trabalho está rodando, o Claude faz verificações de status. Atualmente, o sistema limita essas verificações automáticas quando a sessão está ociosa (máximo de 3 check-ins) para evitar consumo fantasma de tokens. [1] (<https://medium.com/@automation.labs/5-moves-to-make-on-claude-code-before-the-sept-14-limit-cut-b7aae1b11257>)

Erros Críticos: Se o terminal retornar um erro de sistema ou algo que exija intervenção humana real, o Claude limpa o objetivo e para imediatamente para não quebrar nada. [1] (<https://code.claude.com/docs/en/goal>)

O segredo: Como escrever o /goal com segurança

Para garantir proteção total, nunca dê ordens vagas como /goal refatore o projeto. Em vez disso, coloque travas de segurança diretamente no comando. [1] (<https://www.mindstudio.ai/blog/how-to-use-goal-command-claude-code-autonomous-workflows>), [2] (<https://code.claude.com/docs/en/goal>)

1. Sempre dê um critério de saída numérico ou temporal [1]

(<https://code.claude.com/docs/en/goal>)

Adicione um limite de turnos no fim do seu comando. Se ele chegar nesse limite e não terminar, ele para de qualquer forma. [1] (<https://code.claude.com/docs/en/goal>), [2]

(<https://medium.com/@0xmega/claude-code-is-eating-your-budget-7-fixes-that-cut-costs-without-killing-output-f8dbb3b8a04d>)

Exemplo:

```
/goal Migre o módulo de autenticação para a nova API até que 'npm run build' passe sem erros, OU pare após 15 turnos.
```

[1] (<https://code.claude.com/docs/en/goal>)

2. Dê uma linha de chegada baseada em testes ou compilação [1]

(https://www.youtube.com/watch?v=59zh_m2WD-8), [2] (<https://code.claude.com/docs/en/goal>)

A IA precisa de um veredito binário do terminal (passou ou falhou) para ter certeza de que terminou. [1] (<https://code.claude.com/docs/en/goal>)

Exemplo:

```
/goal Crie os testes unitários para a pasta /src/utils. O objetivo estará concluído quando 'jest -coverage' apontar 80% e todos os testes passarem.
```

[1] (https://www.youtube.com/watch?v=59zh_m2WD-8), [2]

(<https://code.claude.com/docs/en/goal>)

3. Defina limites estritos de escopo [1] (<https://www.mindstudio.ai/blog/how-to-use-goal-command-claude-code-autonomous-workflows>), [2] (<https://code.claude.com/docs/en/goal>)

Diga explicitamente o que ele não pode tocar. [1] (<https://www.mindstudio.ai/blog/how-to-use-goal-command-claude-code-autonomous-workflows>), [2] (<https://code.claude.com/docs/en/goal>)

Exemplo:

```
/goal Refatore as queries do arquivo user.service.ts para usar Prisma. Restrição: Não altere nenhum arquivo dentro da pasta /config ou /routes.
```

[1] (<https://code.claude.com/docs/en/goal>)

Dicas de ouro antes de ir dormir:

Sempre use Git: Faça um git commit ou git stash de tudo antes de disparar o comando. Se a IA fizer besteira enquanto você dorme, um simples git reset --hard resolve o seu problema. [1] (<https://horadecodar.com.br/refatorar-claude-code/>)

De olho nos custos: De vez em quando, use o comando /usage ou /cost para ver o quanto a sua sessão atual já consumiu e configurar limites na sua conta da Anthropic. [1]

(<https://code.claude.com/docs/en/costs>), [2] (https://medium.com/@the_infinity/5-claude-code-slash-commands-i-use-every-single-day-c1f3561b7d70), [3] (<https://medium.com/@0xmega/claude-code-is-eating-your-budget-7-fixes-that-cut-costs-without-killing-output-f8dbb3b8a04d>)

Se quiser, compartilhe comigo:

Qual é a tarefa exata (ex: migrar de uma biblioteca X para Y, quebrar um arquivo grande) que você precisa fazer?

Qual comando do seu terminal (como um linter, teste ou build) diz se o código está funcionando?

Rodrigo Sampaio 01:20

Cara, você trouxe um monte de referências aí, mas vamos simplificar. O que você realmente precisa é entender como usar o `/goal` na prática. Quer começar me contando qual tarefa você tem em mente? A gente pode montar juntos um critério de pronto bem seguro e direto.

02:05 Você

Fechado, vamos direto ao ponto! Para a gente montar esse comando `/goal` à prova de loops e 100% seguro, me conta: Qual é a tarefa exata (ex: migrar de uma biblioteca X para Y, refatorar um arquivo gigante) que você quer deixar ele fazendo? Existe algum comando no seu terminal (como um teste, um build ou um linter) que diz com certeza se o código ficou bom e está funcionando? Com isso, a gente desenha o comando perfeito para você colar no terminal e ir dormir em paz. No que você está trabalhando agora? Me conta: Qual é o objetivo da refatoração/migração atual? Qual comando de terminal valida que o código está correto (ex: `npm run test`, `cargo build`, etc.)?

Rodrigo Sampaio 02:10

Boa! Então, me diz aí: o que exatamente você está tentando fazer agora? Tipo, tá migrando uma API, refatorando um módulo, ou algo mais? E qual seria o comando no terminal que valida se o trabalho tá certo, tipo um teste ou um build? Com essas infos, a gente monta um `/goal` redondinho pra você usar.

Exatamente! É exatamente esse o mecanismo. O Avaliador (Evaluator) funciona como um "gerente de controle de qualidade" implacável que roda em segundo plano. O fluxo do loop autônomo funciona assim a cada ciclo:

O Executor age: O modelo principal mexe no código, roda um comando e faz uma alteração.

O Avaliador entra em cena: Um modelo rápido e focado (como o Haiku) assume por um instante. Ele não altera arquivos; ele apenas lê o que foi feito e se pergunta: "O objetivo final que o usuário digitou no /goal foi 100% atingido?"

A tomada de decisão:

Se NÃO foi cumprido: O Avaliador assume o papel do seu "continua" automático. Ele aponta o que ainda está faltando e dispara o próximo turno para o Executor.

Se SIM foi cumprido: Ele encerra o processo na hora e te devolve o controle do terminal.

É por isso que o /goal funciona sozinho: a IA fica conversando com ela mesma (Executor ↔ Avaliador) até que o Avaliador dê o braço a torcer e diga que o trabalho está pronto.

Agora que o funcionamento do Avaliador ficou claro, quer me contar qual é a tarefa de refatoração ou migração que você precisa fazer para a gente montar o /goal com os critérios certos para impressionar esse Avaliador?

Rodrigo Sampaio 03:45

Exatamente, o Avaliador é como um guardião da qualidade! Agora, bora aplicar isso na prática. Qual é a sua tarefa atual? Tá migrando algo, refatorando um módulo, ou o quê? Me fala também qual comando no terminal valida que o trabalho tá correto, tipo um teste ou build. Assim a gente cria um /goal bem alinhado com o que você precisa.

O grande trunfo do `/goal` é exatamente este: destravar o trabalho de fundo (background work) com responsabilidade.

Ao usar esse comando, você deixa de ser um "digitador de 'continua'" e passa a atuar como um diretor de engenharia. Você delega tarefas complexas que podem levar de vários minutos a algumas horas para o Claude Code resolver sozinho enquanto você foca em outra coisa ou vai descansar.

No entanto, autonomia no terminal exige uma contrapartida clara do desenvolvedor:

1. O Critério de Pronto (Definition of Done) precisa ser cirúrgico

Como o Avaliador (Haiku) é literal, se você der um objetivo vago, ele pode manter o loop rodando infinitamente tentando adivinhar o que você quer. O seu critério de sucesso precisa ser binário e incontestável para a máquina.

2. Atenção vermelha ao consumo de tokens

Sessões longas e autônomas consomem muitos tokens. Cada turno acumula o histórico dos turnos anteriores (contexto inflado). Se o Claude Code precisar de 20 ou 30 turnos para resolver uma migração complexa, o custo daquela única sessão pode subir rapidamente.

A regra de ouro é: use a autonomia para tarefas demoradas, mas sempre coloque uma "coleira" de custo e escopo no comando.

Para colocarmos isso em prática com segurança no seu projeto, me diga:

Qual é a tarefa de refatoração ou migração (ex: passar de JavaScript para TypeScript, trocar Axios por Fetch) que você quer delegar para ele rodar em segundo plano?

Qual comando de validação (como `npm run build` ou `pytest`) garante que ele não vai quebrar o sistema?

Com isso, montamos um `/goal` autônomo, mas totalmente responsável e controlado.

Rodrigo Sampaio 04:38

Exatamente! O `/goal` é uma baita ferramenta pra destravar o trabalho pesado de fundo, mas precisa ser usado com critério. Então, qual é a tarefa de refatoração ou migração que você quer mandar pro Claude? E qual comando de validação garante que o código tá funcionando redondo, tipo um `'npm run build'` ou algo assim? Bora construir um comando super eficiente e seguro pra sua necessidade!

04:56 Você

nenhuma

Rodrigo Sampaio 05:00

Sem problemas! Se você não tiver uma tarefa específica agora, posso te dar exemplos de como montar o `/goal` pra diferentes cenários. Mas, se tiver algo em mente, mesmo que seja uma ideia vaga, me conta e a gente ajusta juntos!

pode gerar

Rodrigo Sampaio 05:27

Beleza! Vamos criar um exemplo. Digamos que você queira migrar as funções de uma API de autenticação para uma nova biblioteca. O comando ideal seria algo assim:

```
/goal Migre todas as funções de autenticação do arquivo auth.js para usar a biblioteca OAuth2. O objetivo estará concluído quando 'npm run test' passar sem erros. Limite: Máximo de 10 turnos.
```

Esse comando é direto, tem um critério claro de sucesso ('npm run test' sem erros) e ainda inclui um limite de turnos pra evitar loops infinitos. Que tal? Quer ajustar pra algo mais próximo do seu contexto?

06:20 Você

não desta forma esta boa

Rodrigo Sampaio 06:22

Show! Se essa estrutura funciona pra você, é só adaptar para a tarefa que estiver no seu radar. O segredo tá em ter um objetivo claro, um critério de validação sólido e, claro, um limite de turnos. Se precisar de mais exemplos ou ajustes, é só falar!

FIM DA SESSÃO

Esta transcrição inclui conteúdo gerado por IA e é fornecida para seus propósitos de aprendizado pessoal.